

Deep Learning

Funções de Ativação e Funções de Custo

Lucas Silveira Kupssinskü

Machine Learning Theory and Applications Laboratory
PUCRS

junho de 2026

Visão Geral

1. Onde as ativações entram no MLP
2. Ativações ponto-a-ponto
3. Orientação prática
4. Ativações na camada de saída
5. Construindo funções de custo via MLE
6. Exemplo 1: regressão
7. Exemplo 2: classificação binária
8. Exemplo 3: classificação multiclasse
9. Loss e ativação no PyTorch
10. Tarefa, camada de saída e função de custo
11. Métricas de Avaliação

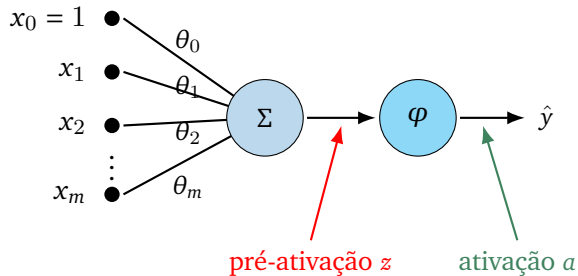
Visão Geral

1. Onde as ativações entram no MLP
2. Ativações ponto-a-ponto
3. Orientação prática
4. Ativações na camada de saída
5. Construindo funções de custo via MLE
6. Exemplo 1: regressão
7. Exemplo 2: classificação binária
8. Exemplo 3: classificação multiclasse
9. Loss e ativação no PyTorch
10. Tarefa, camada de saída e função de custo
11. Métricas de Avaliação

Pré-ativação z vs ativação a

- Cada unidade computa uma soma ponderada das entradas (*pré-ativação*).
- O resultado passa por uma função φ (*ativação*) que determina a saída.
- Toda a riqueza de comportamento da rede depende da escolha de φ .

$$z = \boldsymbol{\theta}^\top \mathbf{x}, \quad a = \varphi(z)$$



Camadas ocultas vs camada de saída

- Em **camadas ocultas**, φ adiciona não-linearidade: sem ela, qualquer rede colapsa em uma transformação afim.
- Em **camada de saída**, φ é escolhida conforme a tarefa (regressão, classificação binária, multiclasse, multilabel).
- As ativações de saída interagem diretamente com a função de custo.

O que esperamos de uma boa ativação

- Contínua e diferenciável (quase em todo lugar): a descida de gradiente exige φ' .
- $\varphi'(z)$ não nula em uma faixa ampla, para que o gradiente não se anule.
- Computação barata: avaliada milhões de vezes por iteração.
- Idealmente sem saturação severa: gradientes preservam ordem de magnitude entre camadas.

Visão Geral

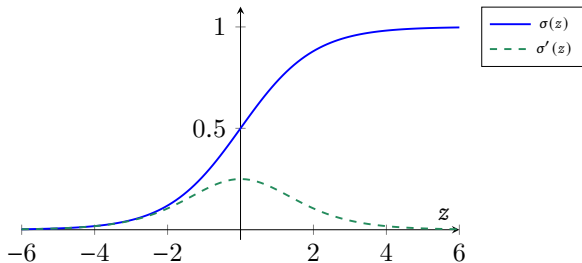
1. Onde as ativações entram no MLP
- 2. Ativações ponto-a-ponto**
3. Orientação prática
4. Ativações na camada de saída
5. Construindo funções de custo via MLE
6. Exemplo 1: regressão
7. Exemplo 2: classificação binária
8. Exemplo 3: classificação multiclasse
9. Loss e ativação no PyTorch
10. Tarefa, camada de saída e função de custo
11. Métricas de Avaliação

Sigmoid

$$\sigma(z) = \frac{1}{1 + e^{-z}}$$

$$\sigma'(z) = \sigma(z) (1 - \sigma(z))$$

- Imagem em $(0, 1)$.
- Derivada máxima 0.25 em $z = 0$.
- Histórica em redes rasas; saturação atrapalha em redes profundas.

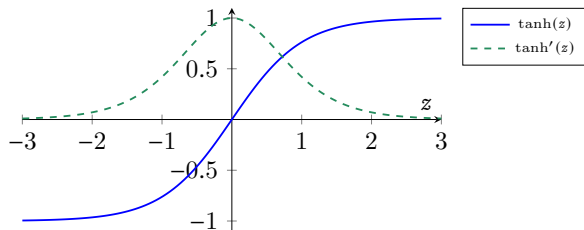


Tangente hiperbólica

$$\tanh(z) = \frac{e^z - e^{-z}}{e^z + e^{-z}}$$

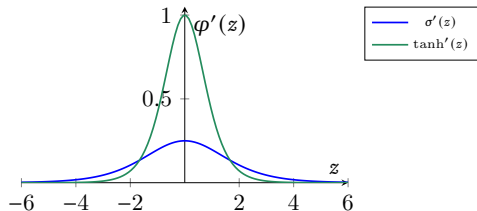
$$\tanh'(z) = 1 - \tanh^2(z)$$

- Imagem em $(-1, 1)$, centrada em zero.
- Mais simétrica que a sigmoid.
- Sofre do mesmo problema de saturação.



Saturação: dissipação de gradiente

- Para $|z|$ grande, $\sigma'(z) \rightarrow 0$ e $\tanh'(z) \rightarrow 0$.
- A regra da cadeia em uma rede profunda multiplica essas derivadas camada a camada.
- Resultado: gradientes vão a zero, camadas iniciais quase não aprendem.
- Esse fenômeno é o *vanishing gradient problem*.

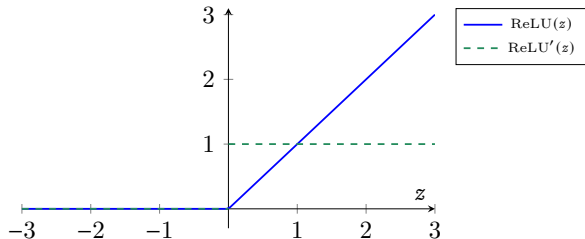


ReLU¹

$$\text{ReLU}(z) = \max(0, z)$$

$$\text{ReLU}'(z) = \begin{cases} 0 & z < 0 \\ 1 & z > 0 \end{cases}$$

- *Rectified Linear Unit*.
- Não satura no lado positivo: gradiente preservado.
- Esparsidade natural (zera o lado negativo).
- Padrão em redes profundas modernas.



¹Vinod Nair e Geoffrey E. Hinton. "Rectified Linear Units Improve Restricted Boltzmann Machines". Em: [ICML](#). 2010, pp. 807–814.

O problema do *dying ReLU*

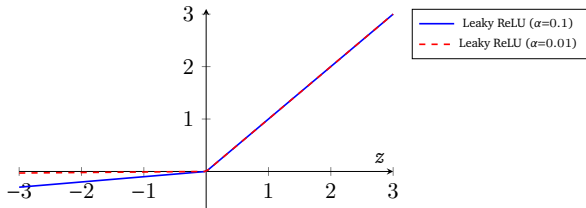
- Se uma unidade entra em $z < 0$ para todos os exemplos do dataset, $\text{ReLU}'(z) = 0$ sempre.
- Sem gradiente passando, os pesos não atualizam: a unidade fica **morta**.
- Em redes muito largas, é comum uma fração significativa das unidades estar morta em algum momento do treinamento.
- Causas usuais: taxa de aprendizado muito alta, inicialização ruim.

Leaky ReLU e PReLU

$$\varphi(z) = \max(\alpha z, z), \quad \alpha \in (0, 1)$$

$$\varphi'(z) = \begin{cases} \alpha & z < 0 \\ 1 & z > 0 \end{cases}$$

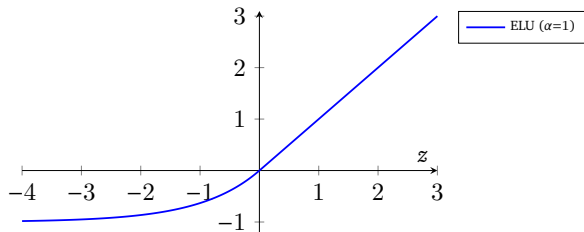
- Leaky ReLU: α fixo (tipicamente 0.01).
- PReLU: α é treinável.
- Mata o problema do *dying ReLU* ao manter gradiente α no lado negativo.



$$\varphi(z) = \begin{cases} z & z \geq 0 \\ \alpha(e^z - 1) & z < 0 \end{cases}$$

$$\varphi'(z) = \begin{cases} 1 & z > 0 \\ \alpha e^z & z < 0 \end{cases}$$

- *Exponential Linear Unit.*
- Suave em torno de 0; sai do regime de unidade morta exponencialmente.
- Empurra a média das ativações para perto de zero.

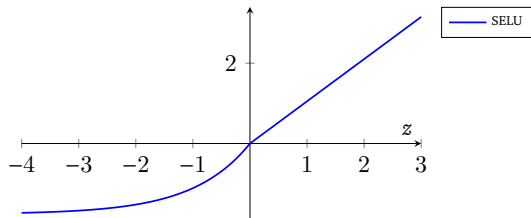


²Djork-Arné Clevert, Thomas Unterthiner e Sepp Hochreiter. "Fast and Accurate Deep Network Learning by Exponential Linear Units (ELUs)". Em: [ICLR](#). 2016.

SELU³

$$\varphi(z) = \lambda \begin{cases} z & z \geq 0 \\ \alpha(e^z - 1) & z < 0 \end{cases}$$

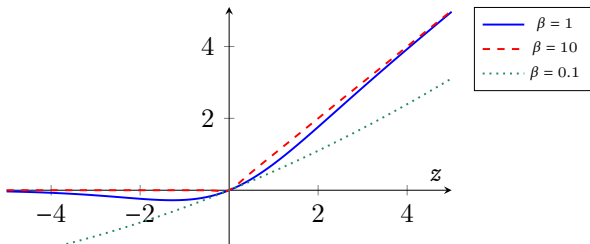
- *Scaled ELU*: ELU multiplicada por uma constante λ .
- Constantes $\alpha \approx 1.6733$ e $\lambda \approx 1.0507$ são escolhidas para que a estatística das ativações se auto-normalize ao longo das camadas.
- Útil quando não usamos *batch norm*.



³Günter Klambauer et al. "Self-Normalizing Neural Networks". Em: [NeurIPS](#). 2017.

$$\varphi(z) = z \sigma(\beta z)$$

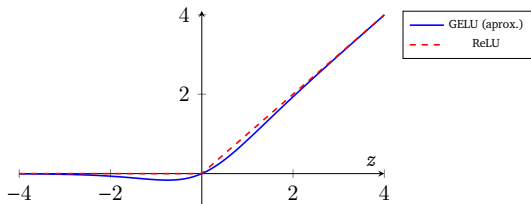
- Não monotônica: passa por um pequeno mínimo negativo antes de subir.
- Diferenciável em todo o domínio.
- Limite $\beta \rightarrow 0$: $\varphi(z) \approx z/2$ (linear).
- Limite $\beta \rightarrow \infty$: $\varphi(z) \rightarrow \text{ReLU}(z)$.
- Encontrada por busca automática em uma família de ativações.



⁴Prajit Ramachandran, Barret Zoph e Quoc V. Le. "Searching for Activation Functions". Em: [arXiv preprint arXiv:1710.05941](https://arxiv.org/abs/1710.05941) (2017).

$$\varphi(z) = z \Phi(z), \quad \Phi(z) = \Pr(Z \leq z), \quad Z \sim \mathcal{N}(0, 1)$$

- *Gaussian Error Linear Unit*.
- Multiplica z pela probabilidade acumulada gaussiana $\Phi(z)$.
- Aproximação prática:
 $\varphi(z) \approx z \sigma(1.702 z)$ (próxima de Swish).
- Padrão em *transformers* modernos (BERT, GPT-2/3, ViT).



⁵Dan Hendrycks e Kevin Gimpel. “Gaussian Error Linear Units (GELUs)”. Em: [arXiv preprint arXiv:1606.08415](https://arxiv.org/abs/1606.08415) (2016).

- Não é uma ativação ponto-a-ponto: combina duas projeções lineares com gating.

$$\text{SwiGLU}(x) = \text{Swish}_\beta(xW_1 + b_1) \odot (xW_2 + b_2)$$

- $\odot$ é o produto elemento a elemento; uma das projeções faz *gating* sobre a outra.
- Substitui o bloco *feed-forward* típico em arquiteturas modernas como LLaMA, Mistral e PaLM.
- Empiricamente: melhora qualidade do modelo com mesmo orçamento de parâmetros e cálculo.

Custo

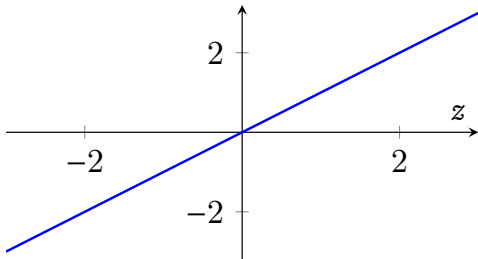
Triplica o número de parâmetros do bloco FFN (duas projeções de entrada em vez de uma), mas o ganho de desempenho compensa.

⁶Noam Shazeer. “GLU Variants Improve Transformer”. Em: [arXiv preprint arXiv:2002.05202](https://arxiv.org/abs/2002.05202) (2020).

Identity (Linear)

$$\varphi(z) = z, \quad \varphi'(z) = 1.$$

- Sem domínio limitado: $\varphi(z) \in \mathbb{R}$.
- Não adiciona não-linearidade: usar Linear em camadas ocultas faz a rede colapsar.
- Faz sentido apenas na **camada de saída**, em tarefas de regressão sem restrição de sinal.



Visão Geral

1. Onde as ativações entram no MLP
2. Ativações ponto-a-ponto
- 3. Orientação prática**
4. Ativações na camada de saída
5. Construindo funções de custo via MLE
6. Exemplo 1: regressão
7. Exemplo 2: classificação binária
8. Exemplo 3: classificação multiclasse
9. Loss e ativação no PyTorch
10. Tarefa, camada de saída e função de custo
11. Métricas de Avaliação

Qual usar em uma camada oculta?

- **Default moderno:** ReLU para CNNs e MLPs simples; GELU para *transformers*.
- **Quando precisa de saída centrada em zero:** tanh em redes recorrentes clássicas.
- **Quando o *dying ReLU* aparece:** trocar para Leaky ReLU ou ELU.
- **Em LLMs grandes:** blocos FFN passam a usar SwiGLU.
- **Sigmoid em camada oculta:** praticamente nunca em redes profundas.

Visão Geral

1. Onde as ativações entram no MLP
2. Ativações ponto-a-ponto
3. Orientação prática
- 4. Ativações na camada de saída**
5. Construindo funções de custo via MLE
6. Exemplo 1: regressão
7. Exemplo 2: classificação binária
8. Exemplo 3: classificação multiclasse
9. Loss e ativação no PyTorch
10. Tarefa, camada de saída e função de custo
11. Métricas de Avaliação

A camada de saída define a interface com a tarefa

- Regressão real: queremos $\hat{y} \in \mathbb{R} \rightarrow$ **Linear**.
- Classificação binária: queremos $\hat{y} \in [0, 1]$ (probabilidade) $\rightarrow$ **Sigmoid**.
- Multilabel: K rótulos independentes, cada um com probabilidade própria $\rightarrow K$ **unidades Sigmoid**.
- Multiclasse exclusiva: K classes mutuamente exclusivas, distribuição categórica $\rightarrow$ **Softmax** (atua sobre a camada inteira).

Sigmoid na camada de saída

- Combinada com BCE (binary cross-entropy) na função de custo.
- Domínio do alvo: $y \in \{0, 1\}$.
- Saída: $\hat{y} = \sigma(z) = \Pr(y = 1 \mid x)$.
- Para multilabel basta uma sigmoid por classe; cada uma se comporta como um classificador binário independente.

Softmax: ativação que age na camada inteira

$$\text{softmax}(z)_i = \frac{e^{z_i}}{\sum_{j=1}^K e^{z_j}}$$

- Cada saída i depende de todas as pré-ativações, não só de z_i .
- Saídas são positivas e somam 1: distribuição de probabilidade.
- Combina com a *categorical cross-entropy* na loss.

z_i	$\text{softmax}(z)_i$
1.8	0.7500
0.7	0.2497
-10.0	0.0003

$$\text{softmax}\begin{pmatrix} 1.8 \\ 0.7 \\ -10.0 \end{pmatrix} \approx \begin{pmatrix} 0.7500 \\ 0.2497 \\ 0.0003 \end{pmatrix}$$

Visão Geral

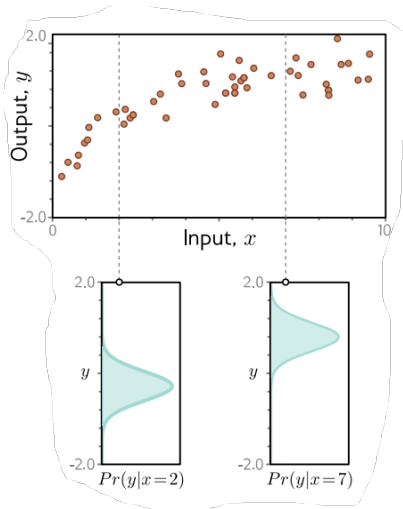
1. Onde as ativações entram no MLP
2. Ativações ponto-a-ponto
3. Orientação prática
4. Ativações na camada de saída
- 5. Construindo funções de custo via MLE**
6. Exemplo 1: regressão
7. Exemplo 2: classificação binária
8. Exemplo 3: classificação multiclasse
9. Loss e ativação no PyTorch
10. Tarefa, camada de saída e função de custo
11. Métricas de Avaliação

Mudança de paradigma

- Até aqui: a rede $f(\mathbf{x}; \boldsymbol{\theta})$ produz uma saída $\hat{y}$.
- A partir de agora: a rede produz **parâmetros ϕ de uma distribuição de probabilidade** sobre y .
- Loss vai medir o quão bem essa distribuição predita explica os dados observados.
- Esse é o ponto de partida da família de *maximum likelihood estimators*.

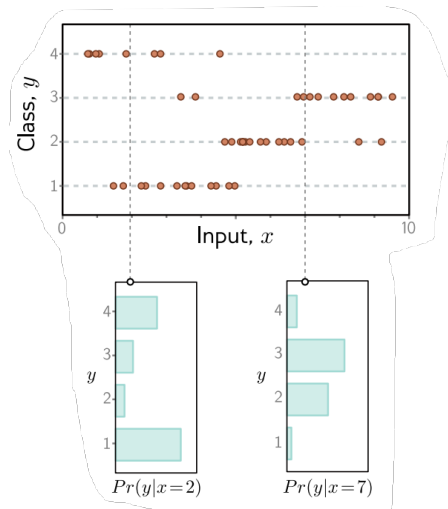
$\Pr(y \mid x)$ em uma tarefa contínua

- Tarefa de regressão: $y \in \mathbb{R}$.
- Para cada x , a rede estima a média de uma gaussiana sobre y .
- As gaussianas verticais à direita ilustram $\Pr(y \mid x=2)$ e $\Pr(y \mid x=7)$.



$\Pr(y | x)$ em uma tarefa discreta

- Tarefa de classificação multiclasse: $y \in \{1, 2, 3, 4\}$.
- Para cada x , a rede estima as probabilidades de cada classe.
- Distribuição categórica (Bernoulli generalizada).



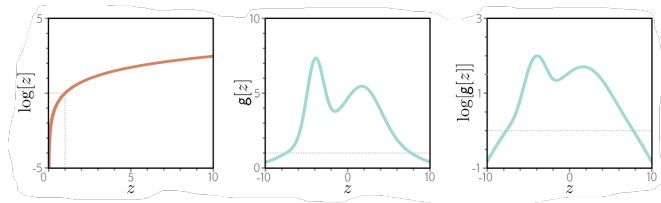
Estimador de máxima verossimilhança

$$\hat{\theta} = \operatorname{argmax}_{\theta} \prod_{i=1}^N \operatorname{Pr}\left(y^{(i)} \mid f(x^{(i)}; \theta)\right)$$

- Maximiza a probabilidade conjunta dos rótulos observados, dada a entrada.
- Hipóteses implícitas:
 - As amostras $(x^{(i)}, y^{(i)})$ são **identicamente distribuídas**.
 - São **independentes** entre si (justifica o produto).
- Em uma frase: *i.i.d.*

Por que aplicar log à verossimilhança?

- $\log$ é monotônica crescente: preserva o ponto de máximo.
- Transforma produto em soma, que é estável numericamente.
- Soma de log-probabilidades ≤ 0 , fácil de manipular.



Log-verossimilhança

$$\begin{aligned}\hat{\boldsymbol{\theta}} &= \operatorname{argmax}_{\boldsymbol{\theta}} \prod_{i=1}^N \Pr\left(y^{(i)} \mid f(x^{(i)}; \boldsymbol{\theta})\right) \\ &= \operatorname{argmax}_{\boldsymbol{\theta}} \log \left[\prod_{i=1}^N \Pr\left(y^{(i)} \mid f(x^{(i)}; \boldsymbol{\theta})\right) \right] \\ &= \operatorname{argmax}_{\boldsymbol{\theta}} \sum_{i=1}^N \log \Pr\left(y^{(i)} \mid f(x^{(i)}; \boldsymbol{\theta})\right)\end{aligned}$$

Convenção: minimizar a *Negative Log-Likelihood*

$$\begin{aligned}\hat{\boldsymbol{\theta}} &= \operatorname{argmax}_{\boldsymbol{\theta}} \sum_{i=1}^N \log \Pr\left(y^{(i)} \mid f(x^{(i)}; \boldsymbol{\theta})\right) \\ &= \operatorname{argmin}_{\boldsymbol{\theta}} \left[- \sum_{i=1}^N \log \Pr\left(y^{(i)} \mid f(x^{(i)}; \boldsymbol{\theta})\right) \right] \\ &= \operatorname{argmin}_{\boldsymbol{\theta}} \mathcal{L}(\boldsymbol{\theta})\end{aligned}$$

- $\mathcal{L}(\boldsymbol{\theta})$ é a **Negative Log-Likelihood** (NLL).
- Diferentes escolhas de distribuição $\Pr(y \mid \cdot)$ produzem diferentes funções de custo conhecidas.

Receita geral para construir uma loss

1. Escolher uma distribuição $\Pr(y \mid \phi)$ apropriada ao domínio de y .
2. Fazer a rede $f(x; \theta)$ predizer os parâmetros ϕ dessa distribuição.
3. Substituir e simplificar a NLL para obter uma loss $\mathcal{L}(\theta)$.
4. Treinar minimizando $\mathcal{L}$ via descida de gradiente.

O que vem agora

Vamos aplicar essa receita em três casos canônicos: regressão (gaussiana), classificação binária (Bernoulli) e classificação multiclasse (categórica).

Visão Geral

1. Onde as ativações entram no MLP
2. Ativações ponto-a-ponto
3. Orientação prática
4. Ativações na camada de saída
5. Construindo funções de custo via MLE
- 6. Exemplo 1: regressão**
7. Exemplo 2: classificação binária
8. Exemplo 3: classificação multiclasse
9. Loss e ativação no PyTorch
10. Tarefa, camada de saída e função de custo
11. Métricas de Avaliação

Passo 1: escolher a distribuição

Para $y \in \mathbb{R}$ a escolha natural é a gaussiana univariada:

$$\Pr(y \mid \mu, \sigma^2) = \frac{1}{\sqrt{2\pi\sigma^2}} \exp\left[-\frac{(y - \mu)^2}{2\sigma^2}\right].$$

- μ é a média (centro da distribuição).
- σ^2 é a variância (largura).

Passo 2: rede prediz a média

$$\Pr(y \mid f(\mathbf{x}; \boldsymbol{\theta}), \sigma^2) = \frac{1}{\sqrt{2\pi\sigma^2}} \exp\left[-\frac{(y - f(\mathbf{x}; \boldsymbol{\theta}))^2}{2\sigma^2}\right].$$

- A rede passa a estimar $\mu = f(\mathbf{x}; \boldsymbol{\theta})$.
- A variância σ^2 é tratada como hiperparâmetro (constante).
- A camada de saída é **Linear** (média pode assumir qualquer valor real).

Passo 3: substituir na NLL e simplificar

$$\begin{aligned}\mathcal{L}(\boldsymbol{\theta}) &= - \sum_{i=1}^N \log \left[\frac{1}{\sqrt{2\pi\sigma^2}} \exp \left(- \frac{(y^{(i)} - f(x^{(i)}; \boldsymbol{\theta}))^2}{2\sigma^2} \right) \right] \\ &= - \sum_{i=1}^N \log \left(\frac{1}{\sqrt{2\pi\sigma^2}} \right) + \sum_{i=1}^N \frac{(y^{(i)} - f(x^{(i)}; \boldsymbol{\theta}))^2}{2\sigma^2} \\ &\propto \sum_{i=1}^N (y^{(i)} - f(x^{(i)}; \boldsymbol{\theta}))^2\end{aligned}$$

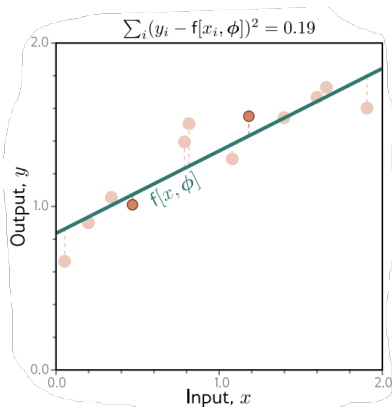
- O primeiro termo não depende de $\boldsymbol{\theta}$; a divisão por $2\sigma^2$ é constante.
- Sobra exatamente o erro quadrático.

Resultado: erro quadrático médio

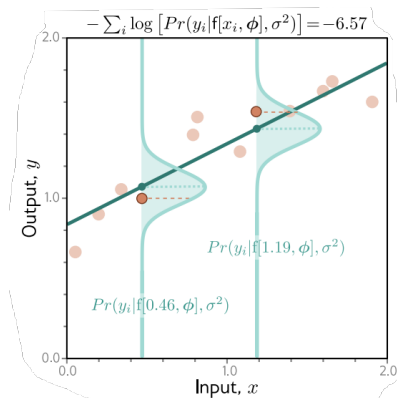
$$\hat{\theta} = \underset{\theta}{\operatorname{argmin}} \sum_{i=1}^N (y^{(i)} - f(x^{(i)}; \theta))^2$$

- Esta é a L_2 **loss** (também chamada de MSE quando dividida por N).
- Sai naturalmente de duas hipóteses: instâncias *i.i.d.* amostradas de uma gaussiana univariada com variância fixa.

Visual: ajuste e gaussianas verticais



Soma dos quadrados dos resíduos.



$-\sum_i \log \Pr(y_i | f(x_i; \theta), \sigma^2).$

Visão Geral

1. Onde as ativações entram no MLP
2. Ativações ponto-a-ponto
3. Orientação prática
4. Ativações na camada de saída
5. Construindo funções de custo via MLE
6. Exemplo 1: regressão
- 7. Exemplo 2: classificação binária**
8. Exemplo 3: classificação multiclasse
9. Loss e ativação no PyTorch
10. Tarefa, camada de saída e função de custo
11. Métricas de Avaliação

Passo 1: distribuição Bernoulli

Para $y \in \{0, 1\}$ usamos a Bernoulli(λ):

$$\Pr(y \mid \lambda) = \begin{cases} 1 - \lambda & y = 0 \\ \lambda & y = 1 \end{cases} = (1 - \lambda)^{1-y} \lambda^y.$$

- $\lambda \in [0, 1]$ é a probabilidade da classe positiva.
- Forma compacta usando expoentes evita o uso explícito de cases.

Passo 2: rede prediz λ via sigmoid

- Saída crua da rede $f(\mathbf{x}; \boldsymbol{\theta}) \in \mathbb{R}$ não respeita $\lambda \in [0, 1]$.
- Aplicamos sigmoid na saída: $\lambda = \sigma(f(\mathbf{x}; \boldsymbol{\theta}))$.

$$\Pr(y \mid \mathbf{x}; \boldsymbol{\theta}) = (1 - \sigma(f(\mathbf{x}; \boldsymbol{\theta})))^{1-y} (\sigma(f(\mathbf{x}; \boldsymbol{\theta})))^y.$$

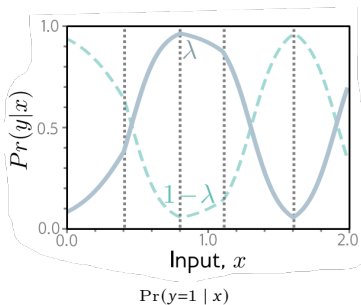
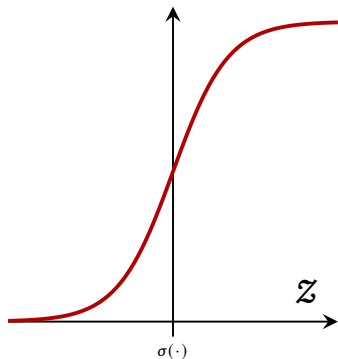
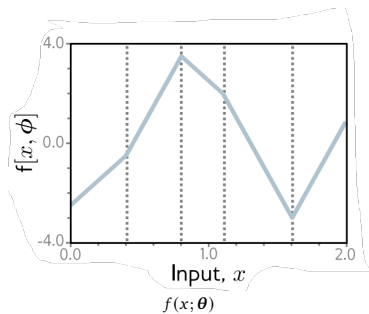
Passo 3: NLL e simplificação

$$\begin{aligned}\mathcal{L}(\boldsymbol{\theta}) &= - \sum_{i=1}^N \log \left[(1 - \sigma(f(x^{(i)}; \boldsymbol{\theta})))^{1-y^{(i)}} (\sigma(f(x^{(i)}; \boldsymbol{\theta})))^{y^{(i)}} \right] \\ &= \sum_{i=1}^N \left[- (1 - y^{(i)}) \log(1 - \sigma(f(x^{(i)}; \boldsymbol{\theta}))) - y^{(i)} \log(\sigma(f(x^{(i)}; \boldsymbol{\theta}))) \right]\end{aligned}$$

Resultado: Entropia Cruzada Binária (BCE)

$$\mathcal{L}_{\text{BCE}}(\boldsymbol{\theta}) = - \sum_{i=1}^N \left[y^{(i)} \log \hat{\lambda}^{(i)} + (1 - y^{(i)}) \log(1 - \hat{\lambda}^{(i)}) \right]$$

Visual: *piecewise* linear $\rightarrow$ sigmoid $\rightarrow \Pr(y | x)$



- A pré-ativação da camada de saída é uma função *piecewise* linear.
- Após a sigmoid, a saída fica em $[0, 1]$ e oscila como uma probabilidade.

Visão Geral

1. Onde as ativações entram no MLP
2. Ativações ponto-a-ponto
3. Orientação prática
4. Ativações na camada de saída
5. Construindo funções de custo via MLE
6. Exemplo 1: regressão
7. Exemplo 2: classificação binária
- 8. Exemplo 3: classificação multiclasse**
9. Loss e ativação no PyTorch
10. Tarefa, camada de saída e função de custo
11. Métricas de Avaliação

Passo 1: distribuição categórica

Para $y \in \{1, 2, \dots, K\}$ (exclusivo), a generalização da Bernoulli é a **categórica**:

$$\Pr(y \mid \lambda) = \prod_{k=1}^K \lambda_k^{[y=k]} \quad \text{com} \quad \sum_{k=1}^K \lambda_k = 1, \lambda_k \geq 0.$$

- $[y = k]$ vale 1 se $y = k$ e 0 caso contrário (Iverson).
- Cada $\lambda_k \in [0, 1]$ e $\sum_k \lambda_k = 1$ formam um vetor de probabilidades.
- Outras notações comuns na literatura: π_k ou p_k no lugar de λ_k .

Passo 2: rede prediz λ via softmax

- Saída crua da rede: $z = f(x; \theta) \in \mathbb{R}^K$ (logits).
- Aplicamos softmax para obter probabilidades:

$$\lambda_k = \text{softmax}(z)_k = \frac{e^{z_k}}{\sum_{j=1}^K e^{z_j}}.$$

- Por construção, λ é um vetor de probabilidades válido.

Passo 3: NLL e simplificação

$$\begin{aligned}\mathcal{L}(\boldsymbol{\theta}) &= - \sum_{i=1}^N \log \left[\prod_{k=1}^K \lambda_k(\mathbf{x}^{(i)}; \boldsymbol{\theta})^{[y^{(i)}=k]} \right] \\ &= - \sum_{i=1}^N \sum_{k=1}^K [y^{(i)} = k] \log \lambda_k(\mathbf{x}^{(i)}; \boldsymbol{\theta}).\end{aligned}$$

Em forma vetorial com $y^{(i)}$ *one-hot*:

$$\mathcal{L}_{\text{CCE}}(\boldsymbol{\theta}) = - \sum_{i=1}^N \sum_{k=1}^K y_k^{(i)} \log(\text{softmax}(f(\mathbf{x}^{(i)}; \boldsymbol{\theta}))_k)$$

Esta é a **Categorical Cross-Entropy** (CCE).

Forma equivalente: log-softmax

Como em cada amostra apenas a classe correta $k^* = y^{(i)}$ contribui (one-hot):

$$\mathcal{L}_{\text{CCE}}(\boldsymbol{\theta}) = - \sum_{i=1}^N \log \left[\frac{e^{z_{k^*}^{(i)}}}{\sum_{j=1}^K e^{z_j^{(i)}}} \right] = - \sum_{i=1}^N \left[z_{k^*}^{(i)} - \log \sum_{j=1}^K e^{z_j^{(i)}} \right].$$

- $\log \sum e^{z_j}$ é o *log-sum-exp*, calculado de forma estável em *frameworks*.
- A derivada de softmax + log será desenvolvida na aula 08 (grafo computacional).

Visão Geral

1. Onde as ativações entram no MLP
2. Ativações ponto-a-ponto
3. Orientação prática
4. Ativações na camada de saída
5. Construindo funções de custo via MLE
6. Exemplo 1: regressão
7. Exemplo 2: classificação binária
8. Exemplo 3: classificação multiclasse
- 9. Loss e ativação no PyTorch**
10. Tarefa, camada de saída e função de custo
11. Métricas de Avaliação

nn.BCEWithLogitsLoss

- Recebe **logits** (pré-ativação).
- Aplica sigmoid + BCE em uma operação numericamente estável (*log-sum-exp trick*).
- Forma recomendada para classificação binária e multilabel.

```
1 import torch
2 import torch.nn as nn
3
4 # Trabalha com logits (saída crua da rede, sem
5   # sigmoid)
6   # Aplica sigmoid + BCE em uma única operação
7   # numericamente estável.
8
9 logits = torch.tensor([2.5, -1.0, 0.3])
10 target = torch.tensor([1.0, 0.0, 1.0])
11
12 loss_fn = nn.BCEWithLogitsLoss()
13 loss = loss_fn(logits, target)
14
15 # Equivalente manual (menos estável):
16 # probs = torch.sigmoid(logits)
17 # loss = -(target * probs.log() + (1 - target)
18   #      * (1 - probs).log()).mean()
19
20 print(loss.item()) # 0.5390...
```

nn.BCELoss

- Recebe **probabilidades** (saída já passada por sigmoid).
- Mais propenso a instabilidade numérica em logits extremos.
- Falha silenciosamente se a entrada não está em (0, 1).

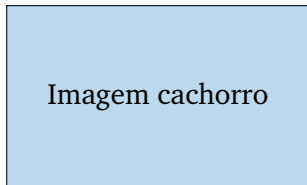
```
1 import torch
2 import torch.nn as nn
3
4 # Trabalha com probabilidades (saída já passada
  por sigmoid).
5 # Numericamente menos estável; falha
  silenciosamente se a entrada
6 # não estiver em (0, 1).
7
8 probs = torch.sigmoid(torch.tensor([2.5, -1.0,
  0.3]))
9 target = torch.tensor([1.0, 0.0, 1.0])
10
11 loss_fn = nn.BCELoss()
12 loss = loss_fn(probs, target)
13
14 # Para classificação multilabel, basta usar a
  mesma loss
15 # em saídas com K unidades sigmoide (uma por
  rótulo).
16
17 print(loss.item()) # 0.5390...
```

Visão Geral

1. Onde as ativações entram no MLP
2. Ativações ponto-a-ponto
3. Orientação prática
4. Ativações na camada de saída
5. Construindo funções de custo via MLE
6. Exemplo 1: regressão
7. Exemplo 2: classificação binária
8. Exemplo 3: classificação multiclasse
9. Loss e ativação no PyTorch
- 10. Tarefa, camada de saída e função de custo**
11. Métricas de Avaliação

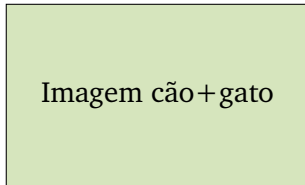
Multiclasse vs Multilabel

- **Multiclasse exclusiva:** o exemplo pertence a **exatamente** uma classe.
- **Multilabel:** o exemplo pode receber **vários** rótulos simultaneamente.



1. Cachorro
2. Gato
3. Rato
4. Humano

Multiclasse: única label correta



1. Cachorro
2. Gato
3. Rato
4. Humano

Multilabel: várias labels possíveis

Cheat sheet: tarefa $\rightarrow$ camada $\rightarrow$ loss

Tarefa	Saída esperada	Camada de saída	Função de custo
Regressão	$\hat{y} \in \mathbb{R}$	1 unidade Linear	MSE (L_2)
Classificação binária	$\hat{y} \in [0, 1]$	1 unidade Sigmoid	BCE
Multilabel (K rótulos)	$\hat{y} \in [0, 1]^K$	K unidades Sigmoid	BCE somada
Multiclasse (K classes)	$\hat{y} \in [0, 1]^K, \sum_k \hat{y}_k = 1$	K unidades Softmax	CCE

- Em PyTorch: `BCEWithLogitsLoss` (binária e multilabel) e `CrossEntropyLoss` (multiclasse) já incorporam softmax/sigmoid.
- Para regressão com restrições (ex.: $\hat{y} \geq 0$), pode-se trocar a Linear por uma SoftPlus na saída e usar uma Poisson (derivação como exercício, ver *lista 02* § “Funções de Custo”).

Visão Geral

1. Onde as ativações entram no MLP
2. Ativações ponto-a-ponto
3. Orientação prática
4. Ativações na camada de saída
5. Construindo funções de custo via MLE
6. Exemplo 1: regressão
7. Exemplo 2: classificação binária
8. Exemplo 3: classificação multiclasse
9. Loss e ativação no PyTorch
10. Tarefa, camada de saída e função de custo
- 11. Métricas de Avaliação**

Métricas $\neq$ função de custo

- A função de custo é otimizada durante o treinamento; precisa ser diferenciável e numericamente estável.
- A métrica é interpretável pelo usuário final; pode ser não diferenciável (acurácia, F1) ou dependente de um *threshold*.
- Quase sempre, a loss é uma **aproximação suave** do que de fato queremos otimizar (a métrica).

Métricas de regressão

- **MSE:** $\frac{1}{N} \sum_{i=1}^N (\hat{y}^{(i)} - y^{(i)})^2$.
- **MAE:** $\frac{1}{N} \sum_{i=1}^N |\hat{y}^{(i)} - y^{(i)}|$.
- R^2 : $1 - \frac{\sum_i (y^{(i)} - \hat{y}^{(i)})^2}{\sum_i (y^{(i)} - \bar{y})^2}$, onde $\bar{y}$ é a média dos rótulos.
- MSE pune grandes desvios mais fortemente; MAE é mais robusto a *outliers*; R^2 é interpretável como variância explicada.

Métricas de classificação

- **Acurácia:** $\frac{TP + TN}{TP + TN + FP + FN}$.
- **Recall:** $\frac{TP}{TP + FN}$.
- **Precision:** $\frac{TP}{TP + FP}$.
- **F_β -score:** $(1 + \beta^2) \frac{P \cdot R}{\beta^2 P + R}$.

		Predito	
		Sim	Não
Real	Sim	TP	FN
	Não	FP	TN

Métricas de classificação: exemplo numérico

Matriz de confusão

		Predito	
		Sim	Não
Real	Sim	8	2
	Não	3	87

100 instâncias, 10 positivas.

$$\text{Acurácia} = \frac{8+87}{100} = 0.95$$

$$\text{Recall} = \frac{8}{8+2} = 0.80$$

$$\text{Precision} = \frac{8}{8+3} \approx 0.727$$

$$F_1 = 2 \cdot \frac{0.727 \cdot 0.80}{0.727+0.80} \approx 0.762$$

Acurácia alta esconde recall mais baixo: classe positiva rara é mais difícil.

Escolhendo a métrica certa

- **Datasets balanceados:** acurácia é razoável.
- **Datasets desbalanceados:** acurácia engana; prefira F1 ou recall específico.
- **Quando o custo do FN é alto** (medicina, fraude): priorize **recall**.
- **Quando o custo do FP é alto** (spam): priorize **precision**.
- **Multiclasse:** macro/micro-F1, matriz de confusão por classe.

Leituras Recomendadas

- Capítulos 5 e 6 de Simon J. D. Prince. Understanding Deep Learning. MIT Press, 2023
- Capítulo 6 de Ian Goodfellow, Yoshua Bengio e Aaron Courville. Deep Learning. MIT Press, 2016. URL: <https://www.deeplearningbook.org/>
- ReLU: Vinod Nair e Geoffrey E. Hinton. “Rectified Linear Units Improve Restricted Boltzmann Machines”. Em: ICML, 2010, pp. 807–814
- GELU: Dan Hendrycks e Kevin Gimpel. “Gaussian Error Linear Units (GELUs)”. Em: arXiv preprint arXiv:1606.08415 (2016)
- Swish: Prajit Ramachandran, Barret Zoph e Quoc V. Le. “Searching for Activation Functions”. Em: arXiv preprint arXiv:1710.05941 (2017)
- SwiGLU: Noam Shazeer. “GLU Variants Improve Transformer”. Em: arXiv preprint arXiv:2002.05202 (2020)

Referências I

- [1] Djork-Arné Clevert, Thomas Unterthiner e Sepp Hochreiter. “Fast and Accurate Deep Network Learning by Exponential Linear Units (ELUs)”. Em: ICLR. 2016.
- [2] Ian Goodfellow, Yoshua Bengio e Aaron Courville. Deep Learning. MIT Press, 2016. URL: <https://www.deeplearningbook.org/>.
- [3] Dan Hendrycks e Kevin Gimpel. “Gaussian Error Linear Units (GELUs)”. Em: arXiv preprint arXiv:1606.08415 (2016).
- [4] Günter Klambauer et al. “Self-Normalizing Neural Networks”. Em: NeurIPS. 2017.
- [5] Vinod Nair e Geoffrey E. Hinton. “Rectified Linear Units Improve Restricted Boltzmann Machines”. Em: ICML. 2010, pp. 807–814.
- [6] Simon J. D. Prince. Understanding Deep Learning. MIT Press, 2023.
- [7] Prajit Ramachandran, Barret Zoph e Quoc V. Le. “Searching for Activation Functions”. Em: arXiv preprint arXiv:1710.05941 (2017).
- [8] Noam Shazeer. “GLU Variants Improve Transformer”. Em: arXiv preprint arXiv:2002.05202 (2020).